

CPF 데이터베이스 표준서

- [CPF 데이터베이스 표준서](#)
 - [1. Owner 와 데이터 경계](#)
 - [2. 공통 원장 패턴](#)
 - [3. 명명](#)
 - [4. Key](#)
 - [5. Version/Concurrency](#)
 - [6. Timestamp](#)
 - [7. Index](#)
 - [8. Transaction/Lock](#)
 - [9. MariaDB/PostgreSQL/Oracle 동일성](#)
 - [10. Migration](#)
 - [11. Expand/Migrate/Contract](#)
 - [12. Drift](#)
 - [13. Large Data](#)
 - [14. Backup · Restore](#)
 - [15. Rollback/Forward Recovery](#)
 - [16. Security](#)
 - [17. DB 변경 Checklist](#)
 - [18. Multi Datasource · Replica](#)
 - [19. Data Quality · Lineage](#)
 - [20. Retention · Purge · Archive](#)
 - [21. SQL 표준](#)
 - [22. 성능 표준](#)
 - [23. cpf-tools DB Lifecycle 정보](#)
 - [24. DB Architecture 원칙](#)
 - [25. Schema · Owner 경계](#)
 - [26. Table · Column 명명 상세](#)
 - [27. Data Type 표준](#)
 - [28. PK · Business Key · Unique Key](#)
 - [29. Optimistic Lock · Expected Version](#)
 - [30. Pessimistic Lock · Lock Order](#)
 - [31. Idempotency Ledger 데이터 모델](#)
 - [32. Attempt · External Result Ledger](#)

- [33. Outbox · Inbox · DLQ Table 표준](#)
- [34. Audit · History Table 표준](#)
- [35. Timestamp · Clock 표준](#)
- [36. Index 설계 표준](#)
- [37. Pagination · 대량 조회](#)
- [38. Partition · Archive · Retention](#)
- [39. 3 Vendor Pack 구조](#)
- [40. Vendor 차이 검토 Matrix](#)
- [41. Migration 파일 표준](#)
- [42. Expand→Migrate→Contract 상세](#)
 - [Expand](#)
 - [Migrate](#)
 - [Switch](#)
 - [Contract](#)
- [43. Drift · Checksum 표준](#)
- [44. Online DDL · Lock 위험](#)
- [45. Backfill 표준](#)
- [46. Read Replica · Consistency](#)
- [47. Backup 정책](#)
- [48. Restore Drill](#)
- [49. Failover · Failback](#)
- [50. Security · Account 표준](#)
- [51. 개인정보 · Masking · 삭제](#)
- [52. Data Quality · Quarantine](#)
- [53. DB Tool 실행 계약](#)
- [54. DB Change 실행 절차](#)
- [55. DB 변경 수용 Checklist](#)
- [56. Source · Evidence 연결](#)
- [57. Canonical Transaction Timeline 데이터 표준 - 07_02](#)

CPF 데이터베이스 표준서

기준 Repository: https://github.com/freeangelsun/202412_01_CPF 기준 Branch: master 문서 콘텐츠 기준 Commit:

3ed676061246c9db3e44f29e254c0393ecca3929 (07_02) 기준일: 2026-08-07 Asia/Seoul 기준 Commit: **현재** master **구현 기준 문서**.

Product Surface · Starter · Tool · EDU 식별자는 아래 기준 Commit 의 Source 정보와 대조한다.

항목	내용
주 독자	DBA · 데이터 모델러 · 개발자 · Migration Owner
이 문서로 완료할 일	논리/물리 모델 · 명명 · Key · Index · Transaction · 3 Vendor Migration · Drift · Backup/Restore · Rollback 표준을 적용한다.
완료 판정	독자가 다른 문서나 Source 역분석 없이 정상 흐름, 오류, 부분 실패, 복구, 감사, 운영 인계를 끝낼 수 있어야 한다.
상태 표현	완료 / 부분 구현 / 미구현 / 미검증 / 실패 / 재확인 필요만 사용한다.
정합성 원칙	실제 Class · API · SQL · Config · 화면 · Permission · Script · Test 를 우선하고 문서와 양방향으로 추적한다.

1. Owner 와 데이터 경계

Table 은 하나의 Owner 가 소유한다. 다른 Domain 은 직접 DML 하지 않고 Public Contract 를 사용한다.

2. 공통 원장 패턴

Current, History, Idempotency, Attempt, Outbox, Inbox, DLQ, Approval Snapshot, Audit, Reconciliation Decision.

3. 명명

Table/Column/PK/FK/UK/Index/Check/Sequence 이름은 길이 제한과 Vendor 차이를 고려한 정규 규칙을 사용한다.

4. Key

Business Key 와 Technical Key 를 구분한다. 외부/분산 식별자는 충돌 · 추적 · 보존기간을 고려한다.

5. Version/Concurrency

상태 변경 Row 에 version/CAS 를 사용하고 stale update 를 차단한다.

6. Timestamp

UTC/지역시간 정책, created/updated/effective from/to, DB/APP clock responsibility 를 명시한다.

7. Index

Query pattern, selectivity, sort, covering, write cost, duplicate index 를 검토한다.

8. Transaction/Lock

Isolation, lock order, deadlock retry, long transaction, batch chunk boundary 를 정의한다.

9. MariaDB/PostgreSQL/Oracle 동일성

Logical type, boolean, identity/sequence, timestamp, pagination, merge/upsert, lock, online DDL 차이를 Vendor Pack 에서 흡수한다.

10. Migration

Versioned forward script, precheck, backup point, apply, verify, data backfill, postcheck, rollback/forward recovery.

11. Expand/Migrate/Contract

Mixed-version 기간에는 old/new application 모두 동작할 수 있는 expand→backfill→switch→contract 순서를 사용한다.

12. Drift

Canonical checksum/version 과 실제 schema/index/constraint/seed 를 비교하고 수동 수정은 승인 없이 정보으로 승격하지 않는다.

13. Large Data

Batch size, lock, undo/WAL/redo, online index, throttling, resumable backfill, checkpoint 를 사용한다.

14. Backup · Restore

Full/incremental/log backup 과 application-consistent restore point 를 정의하고 실제 Restore Drill 로 RPO/RTO 를 측정한다.

15. Rollback/Forward Recovery

Transactional DDL 가능 여부에 따라 rollback script, restore, forward fix 를 선택한다. 데이터 손실 가능성을 명시한다.

16. Security

Runtime/Migration/Read-only/Backup 계정 분리, encryption, masking, audit, least privilege.

17. DB 변경 Checklist

- Owner/Consumer 영향.
- 3 Vendor SQL.
- Lock/성능.
- Mixed Version.
- Backup/Restore.
- Verify/Drift.

- Rollback/Forward Recovery.
- Query/Batch/ADM Consumer Test.

18. Multi Datasource · Replica

Read/Write routing, replica lag, consistency requirement, failover/failback, tenant/domain isolation 을 정의한다.
최신 상태가 필요한 Command 직후 조회는 stale replica 를 피한다.

19. Data Quality · Lineage

Rule ID/Version, dataset/field, source, target, severity, quarantine/correction, approval, reconcile result 를 추적한다.

20. Retention · Purge · Archive

데이터 유형별 retention, purge batch, archive destination, legal hold, 개인정보 deletion, backup 예외를 정의한다. 대량 purge 는 chunk/checkpoint 와 capacity limit 를 사용한다.

21. SQL 표준

Query ID, Owner, parameter/result contract, filter/sort allowlist, Vendor SQL resource 를 사용한다. Java literal SQL 남용을 피하고 SQL 위치와 Consumer 를 추적한다.

22. 성능 표준

대표 데이터 규모에서 plan/index/statistics/partition/slow query/capacity/purge regression 을 검증한다.

23. cpf-tools DB Lifecycle 정보

DB 변경은 sync-database-artifacts.ps1 의 canonical chain 을 통해 3 Vendor Source/Pack/Install/Migration/Rollback/Runtime Query/Verify/Schema Manifest 를 갱신한다. initialize-cpf-database.ps1 은 Platform Pack 을 설치하며 Generated Domain 은 Manifest 기반 전용 초기화 경로를 사용한다. -All 과 개별 selector 를 섞지 않고 한 실행에서 한 Vendor 의미를 유지한다.

24. DB Architecture 원칙

CPF Database 는 **Owner 별 논리 상태, 중앙 Vendor Pack, Runtime Resource Selection** 을 분리한다. Application Module 내부의 우연한 SQL copy 를 정본으로 사용하지 않는다.

Canonical DB Source

```
-> sync-database-artifacts.ps1
  -> vendor/mariadb pack
  -> vendor/postgresql pack
  -> vendor/oracle pack
      -> pack.json / migration / runtime mapper / repository SQL / verify
          -> cpf.db.vendor + cpf.db.resource-root
```

-> Runtime Resolver

cpf.db.vendor 는 mariadb|postgres|oracle 중 하나이며, cpf.db.resource-root 는 선택 Vendor Pack 의 검증된 filesystem root 를 가리킨다. Pack manifest 의 vendor/schemaVersion/status 와 실제 선택값이 맞지 않으면 Runtime 이 다른 Vendor SQL 로 fallback 하지 않는다.

25. Schema · Owner 경계

데이터 종류	Owner 규칙	타 Module 접근
업무 Current/History	업무 Domain Owner	Public Query/Command 또는 승인된 read model
Idempotency/Attempt	해당 Operation Owner	상태 조회 API
Outbox/Inbox/DLQ	Messaging reliability Owner + 업무 correlation	관리 API/ADM
Batch Metadata/Execution	Batch Owner	Batch API/ADM
BZA 조직/권한	BZA Owner	BZA Public contract
Gateway Binding/Apply	Gateway Owner	Gateway Control Plane
Audit	감사 Owner/append-only 정책	검색/Export 권한을 통한 조회

Cross-schema FK 로 Owner 를 강결합하기 전에 장애 · 배포 · Migration 독립성을 검토한다. 다른 Owner 에 대한 직접 DML 은 금지한다.

26. Table · Column 명명 상세

Table 은 기능/Owner 가 식별되는 이름을 사용하고 History/Attempt/Audit 등 역할을 suffix 또는 일관된 naming 으로 표현한다. Column 은 동일 개념에 서로 다른 이름을 만들지 않는다.

권장 공통 의미:

- *_id: 기술/업무 식별자.
- *_code: 관리되는 코드값.
- *_status 또는 *_status_code: 상태.
- version/row_version: optimistic concurrency.
- operation_id, idempotency_key, request_hash: 재실행/응답 유실 추적.
- created_at, updated_at, effective_from, effective_to: 시간 의미 분리.
- created_by, updated_by, reason: 감사 가능 변경.

실제 기존 Schema naming 이 다르면 무조건 Rename 하지 않고 Migration/호환 비용을 검토한다.

27. Data Type 표준

논리 타입	표준 고려사항
Identifier	최대 길이와 정렬/Index 비용, UUID/text 여부
Money	정밀 Decimal; floating point 금지
Boolean	Vendor Pack 에서 논리 의미 통일
Timestamp	Instant/Timezone 의미와 precision
JSON	검색/Index 요구가 있을 때 Vendor 차이 검토
Large text/binary	Inline 저장 상한과 외부 Artifact 분리
Enum/Status	Check/Reference table/코드 원장 선택

Java 타입과 DB 타입의 precision/scale/nullability 를 Contract Test 한다.

28. PK·Business Key·Unique Key

Technical PK 는 Row identity 를 위한 값이고 Business Key 는 업무 중복 판정 값이다. Idempotency Key 와 Business Key 도 동일하지 않을 수 있다.

Unique Constraint 는 Application precheck 를 대체하는 최후의 동시성 Guard 로 사용한다. Duplicate 예외는 사용자에게 DB vendor error raw message 를 노출하지 않고 CPF Conflict 의미로 변환한다.

29. Optimistic Lock·Expected Version

상태 변경은 다음 패턴을 기준으로 한다.

```
UPDATE owner_table
  SET status = :next_status,
      version = version + 1,
      updated_at = :now
 WHERE id = :id
    AND version = :expected_version;
```

영향 행이 0 이면 stale/not-found 를 구분해 최신 Row 를 재조회한다. UI 의 Expected Version 을 숨기더라도 Server 내부 CAS 를 생략하지 않는다. Calendar, Runtime Control, Gateway, Batch 위험 조치 등 실제 화면에서 Version 이 노출되는 경우 Guide 와 일치시킨다.

30. Pessimistic Lock · Lock Order

Pessimistic Lock 은 필요한 좁은 Critical Section 에만 사용한다. 다중 Table Lock 은 고정 순서를 문서화한다. Deadlock 은 무제한 Retry 하지 않고 Transaction 전체를 rollback 한 뒤 bounded retry 또는 사용자 재판정을 사용한다.

Batch 대량 처리에서 긴 Transaction 으로 전체 파일/Job 을 묶지 않는다.

31. Idempotency Ledger 데이터 모델

Ledger 는 최소 다음 정보를 식별 가능하게 유지한다.

Column 의미	목적
key/scope	중복 요청 경계
request_hash	같은 Key 의 payload conflict 판정
operation_id	사용자 요청 추적
status	REQUESTED/RUNNING/확정/UNKNOWN 등 실제 구현 상태
result_ref/snapshot	동일 요청의 deterministic response
created/updated/ expires	보존 · 재처리 판단

Retention 이 만료된 뒤 동일 Key 가 다시 들어올 수 있는 경우 업무 중복 위험을 별도로 정의한다.

32. Attempt · External Result Ledger

외부 HTTP/Broker/File/Gateway/Notification 의 실제 시도는 Operation 과 분리한다. Attempt No, Target, Started/Completed, Timeout stage, External tracking, Response hash, Error, Retry decision 을 보존한다. 응답 유실 후 Side Effect 여부가 불명확하면 UNKNOWN_RESULT 와 Reconciliation Decision 으로 연결한다.

33. Outbox · Inbox · DLQ Table 표준

- Outbox Row 는 업무 Commit 과 원자적으로 생성한다.
- Claim/Lease/Fencing 으로 다중 Publisher 가 같은 Row 를 동시에 소유하지 않게 한다.
- Inbox 는 Event ID/Consumer Scope 로 중복 업무 반영을 막는다.
- DLQ 는 원본 Event 정보와 Failure/Attempt/Audit 를 보존한다.
- Replay 는 새 Event 로 위장하지 않고 원본과 재처리 Operation 을 연결한다.

34. Audit · History Table 표준

Audit 는 Actor, Action, Target, Reason, Approval, Operation, Result, Timestamp 를 추적한다. Before/After Snapshot 에 PII/Secret 을 그대로 복사하지 않고 Mask/Redact/Hash 정책을 적용한다. History 는 업무 상태 이력이고 Audit 는 누가 왜 바꿨는지에 초점을 두므로 한 Table 로 무조건 합치지 않는다.

35. Timestamp·Clock 표준

- 저장 기준 Clock 과 표시 Timezone 을 분리한다.
- DB CURRENT_TIMESTAMP 와 Application Clock 을 섞을 경우 정렬/Lease/Expiry 의미를 검증한다.
- Effective period 는 start inclusive/end exclusive 같은 경계 규칙을 정한다.
- Scheduler/Approval/Secret/Session expiry 는 Clock Skew 영향을 검토한다.
- 테스트는 고정 Clock 또는 제어된 시간으로 경계값을 재현한다.

36. Index 설계 표준

Index 는 실제 Query 와 Sort 를 근거로 만든다. 다음을 기록한다.

1. 대상 Query/Operation ID.
2. Predicate 와 선택도.
3. Sort/Range.
4. 예상 Row 수와 증가율.
5. Write 증폭/Storage 비용.
6. Vendor 별 실행계획 차이.

중복 Index, 낮은 선택도의 단일 Index, 무제한 wildcard 검색을 자동 추가하지 않는다.

37. Pagination·대량 조회

운영 화면과 Export 는 상한 없는 전체 조회를 피한다. Deep paging 이 문제가 되는 원장은 keyset/seek 또는 Export Job 을 검토한다. limit 상한은 API/DB 양쪽에서 강제한다.

38. Partition·Archive·Retention

대규모 원장은 생성일/업무일/상태 등 실제 purge/access 패턴에 맞춰 Partition 여부를 결정한다. Partition 은 성능을 자동 보장하지 않으므로 Query pruning 과 local/global index 비용을 Vendor 별 검증한다.

Retention 삭제는 다음 순서를 사용한다.

```
Eligibility -> Legal Hold/Exception -> Chunk select -> Delete/Archive -> Verify count -> Checkpoint -> Repeat
```

39. 3 Vendor Pack 구조

현재 Runtime 은 중앙 Vendor Pack 을 사용한다. pack.json 은 vendor, schemaVersion, status 필수 필드를 검증하고 schemaVersion 1 을 기준으로 한다. 지정 root 바깥으로 나가는 relative path, symbolic link escape 를 허용하지 않는다.

CpfSqlResourceResolver 는 선택 Vendor Pack 의 runtime/<domain>/mybatis, runtime/<domain>/repository, migration 경로를 사용한다. 선택 resource 가 비어 있으면 다른 Vendor 또는 module-local SQL 로 fallback 하지 않고 실패한다.

40. Vendor 차이 검토 Matrix

항목	MariaDB	PostgreSQL	Oracle	표준화 방법
Identity/ Sequence	구현 차이	구현 차이	구현 차이	Vendor DDL Pack
Boolean	표현 차이 가능	native	표현 차이 가능	Logical mapping
Upsert/Merge	문법 차이	문법 차이	MERGE 계열	Repository SQL 분리
Pagination	LIMIT 계열	LIMIT/ OFFSET	FETCH/ROWNUM 계열	Query Pack
Lock/Skip Locked	지원 문법 차이	지원 문법 차이	지원 문법 차이	Batch/Lease SQL 별도 검증
Online DDL	기능/옵션 차이	기능/lock 차이	기능/edition 차이	Migration runbook

표는 논리 차이만 설명하며 실제 지원 문법은 현재 Vendor SQL Source 를 정본으로 한다.

41. Migration 파일 표준

Migration 은 파일명/Version/Description 에서 순서를 식별할 수 있어야 한다. 한 번 적용된 Migration 을 편의상 수정해 checksum drift 를 숨기지 않는다. 수정이 필요하면 정해진 migration/repair 정책과 새 Version/Forward Fix 를 사용한다.

각 Migration 은 가능한 범위에서 다음 Evidence 를 가진다.

- Precheck query.
- 변경 대상/예상 lock.
- Apply SQL.
- Data backfill/Chunk.
- Verify query.
- Rollback 또는 Forward Fix.
- Mixed Application Version 영향.

42. Expand→Migrate→Contract 상세

Expand

Old/New Application 이 모두 동작하도록 nullable/additive schema 또는 호환 view/interface 를 추가한다.

Migrate

Backfill 은 resumable chunk/checkpoint 로 수행한다. 대량 update 는 WAL/Redo/Undo, lock, replica lag 를 관찰한다.

Switch

Application/Feature Flag/Config 를 새 schema 의미로 전환하고 read/write consistency 를 검증한다.

Contract

Old Version 이 제거됐다는 Evidence 후 구 Column/Constraint 를 정리한다. Contract 를 Deploy 와 같은 시점에 강제하지 않는다.

43. Drift · Checksum 표준

Drift 는 Migration History 만 비교하지 않고 Schema/Table/Column/Index/Constraint/Seed/Pack Hash 등 제품이 요구하는 실제 Surface 를 비교한다. 수동 DDL 이 발견되면 그 상태를 정보으로 자동 승격하지 않는다.

Detect -> Freeze risky change -> Identify owner/change ticket -> Compare canonical ->
Decide forward-fix or restore -> Apply -> Verify -> Record audit/evidence

44. Online DDL · Lock 위험

DDL 전에 예상 lock mode, metadata lock, rewrite 여부, disk temp, replica lag 를 확인한다. 큰 Table 의 Column type 변경/Index 생성/Constraint validation 은 Vendor 별 online option 과 maintenance window 를 검토한다.

DDL 중단 후 상태가 명확하지 않으면 동일 SQL 을 바로 재실행하지 않고 Catalog/Schema 상태를 먼저 조회한다.

45. Backfill 표준

Backfill Job 은 PK/Business Key 범위를 stable 하게 자르고 Checkpoint 를 저장한다. 재실행 시 이미 처리된 Row 를 중복 Side Effect 로 만들지 않게 조건을 둔다. Success Count 만 보지 않고 Source/Target Count, Reject/Quarantine, Checksum 또는 업무 대사를 사용한다.

46. Read Replica · Consistency

Read Replica 를 사용하는 경우 Replica Lag 를 운영 지표로 본다. Command 직후 최신 상태 확인, Expected Version 확인, Approval 실행 결과 등 strong read 가 필요한 경로를 Replica 로 보내지 않는다. Failover 후 Read/Write endpoint 전환과 stale DNS/connection pool 영향을 검증한다.

47. Backup 정책

Backup 정책은 Engine 기능명보다 다음 Evidence 를 남긴다.

- Backup ID/Time/Database/Schema.
- Full/Incremental/Archive/Log 범위.
- Encryption/Key Reference.
- Checksum/Validation.
- Retention/Offsite copy.
- Restore test 대상과 결과.

Backup 생성 성공 메시지만으로 복구 가능성을 판정하지 않는다.

48. Restore Drill

Restore 는 격리 환경에서 수행하고 다음을 확인한다.

1. 목표 Restore Point.
2. DB Engine/Version/Extension/Character set.
3. Backup/Log chain 무결성.
4. Schema/Row/Key Count 검증.
5. Application startup/migration state.
6. Audit/Idempotency/Outbox 같은 운영 원장 정합성.
7. RPO/RTO 실제 측정.

이 문서 작성 세션에서는 Live Restore Drill 을 수행하지 않았으므로 Runtime 검증 상태는 미검증이다.

49. Failover · Failback

Primary 장애에서 Replica 승격 후 Application connection, Session/Transaction retry, Scheduler leader, Broker/Outbox publisher 의 중복 소유를 확인한다. Failback 은 단순 DNS 복귀가 아니라 데이터 divergence/re-seed 와 write freeze 범위를 포함한다.

50. Security · Account 표준

Account	권한 원칙
Runtime	필요한 DML/Execute 만, DDL 최소화
Migration	승인된 DDL/DML, 별도 Secret
Read-only/ Support	Masking/Scope 와 SELECT 제한
Backup	Backup/Restore 에 필요한 권한만
Monitoring	Catalog/Health 최소 권한

Password/Key 원문을 Migration Script 나 Evidence 에 넣지 않는다. Secret Reference 와 Runtime provider 를 사용한다.

51. 개인정보 · Masking · 삭제

PII Column 은 분류/Masking/Raw access permission/Retention 을 문서화한다. 운영 화면의 Raw PII 는 별도 Permission 과 사유/Audit 를 요구한다. 개인정보 삭제는 업무/법적 보존과 Backup 의 별도 정책을 구분한다.

52. Data Quality · Quarantine

Rule ID/Version, Source Record, Validation Result, Quarantine ID, Corrected Snapshot, Expected Version, Approval, Replay/Reconcile 결과를 연결한다. 정정 승인 응답이 유실됐다고 동일 Payload 를 새 Idempotency Key 로 재요청하지 않는다.

53. DB Tool 실행 계약

Tool	목적	안전 조건
sync-database-artifacts.ps1	3 Vendor Source/Pack/Schema/Query/Checksum 동기화 · 검증	선택 Vendor/All 의미 혼용 금지, generated diff 확인
initialize-cpf-database.ps1	Platform Pack 신규 설치 · 검증	대상 Vendor/DB/계정/backup point 확인
Generated Domain init path	Domain Manifest 기반 초기화	Owner Domain 과 Vendor Manifest 일치
full-product verification	DB 관련 정적/실행 Gate 집계	SKIPPED 를 PASS 로 승격 금지

54. DB Change 실행 절차

1. Canonical Source 와 기준 Commit 을 기록한다.
2. Owner/Consumer/Query/Batch/ADM 영향도를 작성한다.
3. MariaDB/PostgreSQL/Oracle 변경본을 생성한다.
4. Precheck 와 예상 Lock/용량을 검토한다.
5. Backup/Restore 또는 Rollback/Forward Fix 경로를 확정한다.
6. Migration 을 격리 환경에서 적용한다.
7. Schema/Index/Constraint/Seed/Runtime Query 를 Verify 한다.
8. Mixed Version 을 검증한다.
9. 실패/중단 Fault 를 재현하고 재실행/Forward Fix 를 검증한다.
10. Tool output, SQL hash, Test result, 미검증 범위를 Evidence 에 기록한다.

55. DB 변경 수용 Checklist

- Table/Data Owner 가 명확하다.
- 다른 Domain 직접 DML 을 추가하지 않았다.
- 3 Vendor resource 가 같은 논리 계약을 구현한다.
- cpf.db.vendor/cpf.db.resource-root 기준 Pack 검증이 가능하다.
- PK/UK/FK/Check/Index 목적이 Query 와 연결된다.

- Expected Version/Idempotency/Attempt 원장이 필요한 곳에 있다.
- 대량 변경의 Chunk/Checkpoint/Lock/Storage 계획이 있다.
- Expand/Migrate/Contract 의 mixed-version 기간이 정의됐다.
- Backup/Restore 또는 Forward Fix 가 실제로 실행됐는지 상태를 구분했다.
- DBA/개발/Batch/ADM Consumer Test 결과를 구분 기록했다.

56. Source · Evidence 연결

DB 정합성은 PROPERTY_LEAF_CATALOG.csv, SOURCE_TRACE_MATRIX.csv, 05 플랫폼 운영 매뉴얼의 DB Lifecycle, 01/02 의 개발 · Batch DB 절차와 함께 검토한다. 현재 243 Runtime leaf 중 DB 선택/Resource/DataSource 관련 Key 도 해당 Source Path 와 함께 기록되어 있다.

현재 문서 패키지의 정적 대조가 Live MariaDB/PostgreSQL/Oracle 실행을 의미하지 않는다. Live DB3 Migration/Upgrade/Rollback/Drift/Restore 결과는 별도 Runtime Evidence 가 있어야 완료로 판정한다.

57. Canonical Transaction Timeline 데이터 표준 - 07_02

Transaction/Segment/Attempt/Remote Call/Batch Execution 을 원 transactionId 로 연결하는 Schema 와 Index 를 설계한다. 대량 데이터에서도 transactionId 단일 조건의 Timeline 조회를 지원하도록 선두 Index 와 시간/Segment 보조 조건을 실제 실행계획으로 검증한다.

Append 중복, 멍등 처리, Partial Log Persistence, DB 장애 후 재기록, Retention/Partition/Archive/Purge 를 명시하며 Audit Record 는 append-only/tamper-evident 요구를 별도 적용한다. MariaDB/PostgreSQL/Oracle 3 종 DDL 의미가 같아야 한다. 실제 대량 성능과 장애 복구는 이번 문서 작업에서 미검증이다.